CIVICSIFT AI

Week 3 – Implementing and Evaluating Text Classification Models

NLP Internship Report | YuvaIntern

Submitted by: Riya Desai

1. Introduction

CivicSift AI is an NLP-based project concept for supporting the triage of citizen complaints. The Week 3 task focused on implementing and evaluating supervised text-classification models using a public complaint dataset. The objective was to compare baseline algorithms, understand their performance, and identify limitations that should guide further development.

2. Objective

The objective was to prepare complaint-related text, convert it into numerical features using TF-IDF, train multiple classification algorithms, evaluate their performance on a held-out test set, and build a simple prediction script for new text.

3. Dataset and Preparation

A sample of NYC 311 service-request records was retrieved from the NYC Open Data Socrata API (dataset endpoint: https://data.cityofnewyork.us/resource/erm2-nwe9.csv). The selected fields were complaint_type and descriptor. The descriptor field was used as the input text and complaint_type as the target label.

A total of 369 records were initially fetched. After removing records with missing required values and preparing the text and target columns, 203 usable rows remained across 24 complaint categories. The data was split using a stratified 75:25 train-test split, resulting in 152 training records and 51 test records.

Important data limitation: the descriptor field often contains short, structured descriptions rather than complete narratives written by citizens. The sample is small and represents NYC English-language service requests; it does not establish multilingual performance or suitability for deployment in another city.

4. Methodology

4.1 Text representation

TF-IDF (Term Frequency–Inverse Document Frequency) was used to represent text numerically. The vectorizer used English stop-word removal and unigrams plus bigrams (ngram_range=(1, 2)). The vectorizer was included inside each scikit-learn pipeline so that it was fitted only on training data, reducing the risk of data leakage.

4.2 Classification algorithms

Three supervised classifiers were trained and compared:
• Multinomial Naive Bayes – a probabilistic baseline commonly used for text classification.
• Logistic Regression – a linear classifier that estimates class probabilities.
• Linear Support Vector Classifier (Linear SVM) – a linear margin-based classifier.

4.3 Evaluation metrics

The models were evaluated on the held-out test set using accuracy, weighted precision, weighted recall, and weighted F1-score. Weighted metrics account for class support when combining per-class scores.

5. Results

Model	Accuracy	Weighted Precision	Weighted Recall	Weighted F1
Multinomial Naive Bayes	37.25%	28.11%	37.25%	30.14%
Logistic Regression	43.14%	40.23%	43.14%	37.83%
Linear SVM	49.02%	50.33%	49.02%	47.13%

Among the three tested models, Linear SVM achieved the strongest results on this test split, with 49.02% accuracy and a weighted F1-score of 47.13%. Logistic Regression ranked second, while Multinomial Naive Bayes produced the lowest scores. These results are preliminary because the dataset is small and the category distribution is uneven.

6. Prediction Demonstration

Input complaint	Model prediction	Observation
Vehicles are parked illegally	Illegal Parking	Correct for this example
There is a pothole on the road	Consumer Complaint	Incorrect prediction
Loud music is disturbing residents	Noise - Commercial	Related category, but not an exact match

The examples demonstrate that the model can return predictions for new text, but its output should not be treated as consistently correct. Errors are expected with limited training data, overlapping categories, and short text descriptions.

7. Implementation and Project Files

The implementation was organized into Python scripts and supporting folders:
• 01_data_preparation.py – retrieves and prepares the sample dataset.
• 02_train_models.py – creates TF-IDF/classifier pipelines, trains the models, evaluates them, and saves model artifacts and comparison results.
• 03_predict_complaint.py – loads a saved model and predicts a category for entered complaint text.
• data/nyc_311_complaints.csv – prepared dataset sample.
• models/ – saved joblib model pipelines (Naive Bayes, Logistic Regression, and Linear SVM).

• results/model_comparison.csv and model_comparison.png – evaluation summary and visualization.

The Python virtual environment folder (.venv) is a local development dependency folder and should not be committed to the GitHub repository.

8. Challenges and Limitations

- Small sample size: only 203 records remained after preparation, limiting generalization.
- Multiple categories: 24 complaint labels create a challenging multi-class task, especially where some labels have few examples.
- Class imbalance: weighted metrics are useful, but per-class evaluation and a confusion matrix would provide more detail.
- Short input text: the descriptor field may not represent the full context of a citizen complaint.
- Geographic and language scope: NYC English-language data does not demonstrate performance on Indian civic complaints or multilingual text.
- Limited tuning: this experiment used a basic train-test evaluation and did not include extensive hyperparameter search or cross-validation.
- Not production-ready: predictions require human review and further testing before any real-world use.

9. Future Improvements

- Collect a larger, more representative, and appropriately licensed dataset.
- Evaluate per-class precision, recall, F1-score, and a confusion matrix.
- Use cross-validation and hyperparameter tuning for a more reliable comparison.
- Investigate class imbalance and consider suitable resampling or class-weight strategies.
- Test on full complaint narratives and data from the intended deployment region.
- Explore multilingual preprocessing and suitable multilingual transformer models in later project stages.
- Keep a human-in-the-loop review process for uncertain or high-impact classifications.

10. Conclusion

The Week 3 task implemented a complete baseline text-classification workflow for CivicSift AI, from data preparation and TF-IDF feature extraction to model training, evaluation, and interactive prediction. Linear SVM performed best among the three tested models on the available test split. However, the results are preliminary and the current model is a prototype rather than a deployment-ready civic complaint system. The findings provide a baseline for subsequent NLP tasks and further experimentation.

11. References

- NYC Open Data – 311 Service Requests dataset: https://data.cityofnewyork.us/Social-Services/311-Service-Requests-from-2010-to-Present/erm2-nwe9
- scikit-learn documentation – TfidfVectorizer: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- scikit-learn documentation – model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html
- scikit-learn documentation – text feature extraction: https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction